

The support for FTP over TLS (Transport Layer Security) allows you to run a FTP connection securely through TLS encryption and certificate authentication support. The `FTP_TLS` class inherits from `FTP`, defining these additional objects (see Table 2).

Table 2

FTP TLS objects	
<code>FTP_TLS.ssl_version</code>	This specifies the SSL version to use (defaults to <code>TLSv1</code>).
<code>FTP_TLS.auth()</code>	Sets up a secure control connection by using TLS or SSL, depending on what is specified in the <code>ssl_version()</code> attribute.
<code>FTP_TLS.prot_p()</code>	Sets up a secure data connection.
<code>FTP_TLS.prot_c()</code>	Sets up a clear text data connection.

This is a simple FTP client program:

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import ftplib
import os
import socket

path = '/pub/Files'
filename = 'old_template.tar.gz'
host = "127.0.0.1"

# get local working directory
f = os.popen("pwd")
local = f.read()
def main():
    try:
        ftp = ftplib.FTP(host)
    except (socket.error, socket.gaierror) as e:
        print 'ERROR: cannot reach "%s"' % host
        return
    print '*** Connected to host "%s"' % host
    try:
        ftp.login()
    except ftplib.error_perm:
        print 'ERROR: cannot login anonymously'
        ftp.quit()
        return
    print '*** Logged in as "anonymous"***'
    # anonymous user root directory
    print ftp.pwd()
    try:
        ftp.cwd(path)
    except ftplib.error_perm:
        print 'ERROR: cannot CD to "%s"' % path
        ftp.quit()
        return
    print '*** Changed to "%s" folder' % path
```

```
try:
    ftp.retrbinary('RETR %s' % filename,
        open(filename, 'wb').write)
except ftplib.error_perm:
    print 'ERROR: cannot read file "%s"' % filename
    os.unlink(FILE)
else:
    print '*** Downloaded "%s" to "%s"' % (filename, local)
    ftp.quit()
    return
if __name__ == '__main__':
    main()
```

The above is an interactive example of an FTP client. The initial lines import modules and subsequent lines set variables with constants.

```
f = os.popen("pwd")
local = f.read()
```

The above code fragment retrieves the current working directory of the client machine.

In the main function, it performs the following operations:

- 1) Connects to the host 127.0.0.1.
- 2) Logs in as an anonymous user.
- 3) Changes the current working directory to '/pub/Files' using `cwd()`.
- 4) Downloads the file 'old_template.tar.gz' from the server, stores it in the local directory and prints an appropriate message.

At every step mentioned above we check for exceptions; if these are caught, we print an error message. Finally, the main method is called.

Though the above example performs a specific task, this is not the limit for what *ftplib* can do. We can use *ftplib* to create a custom application, but you can just as well create an interactive command-line application. This article gives you a good idea about the Python FTP library and the methods available with it. Python is a powerful language. It is always good to know about its capabilities. Some day you may find this library useful and helpful in leveraging the operations available to you. **END** 🐧

References

- [1] http://en.wikipedia.org/wiki/File_Transfer_Protocol
- [2] <http://docs.python.org/library/internet>

By: Chetan Tarale

The author is an independent programmer and is interested in application security, reverse engineering and penetration testing. He also loves to explore GNU/Linux and emerging open source technologies.